

AI コードレビュー・オーケストレーション構成のコスト-欠陥検出性能比較

第 33 回汎用人工知能研究会 (SIG-AGI)

高原 大明

2026/08/21

北陸先端科学技術大学院大学

先端科学技術研究科 先端科学技術専攻

人間情報学研究領域

社会的信号処理・マルチモーダルインタラクション研究室

背景と課題

ベンチマーク構築

実験設計

結果

まとめ

1. 構成の優位は，計算量と分離しないと確かめられない

汎用人工知能には，限られた計算資源の下で多様な課題を効率的に解く能力が要る．エージェントを何体・何回動かすかという資源配分は，その中心的な設計問題である．

LLM コードレビューの構成比較は計算量（コスト）を統制しておらず [1, 2]，観測された差が構造由来か計算量由来か切り分けられない．一般推論では予算を揃えると優位が消える報告があるが [3, 4, 5]，コードレビューには誤りの非対称性があり，同じ結論が成り立つかは未検証である．

レビューにおける 2 種類の誤りと，その対処コスト

誤り	内容	対処に要するもの
誤指摘 (FP)	存在しない欠陥を挙げる	レビューアが却下すれば済む
見逃し (FN)	実在する欠陥を挙げない	発見に人手のコードレビュー

1. 予算帯によって、最適な構成は入れ替わる

RQ1 同等コスト水準で、構成間に欠陥検出率・偽陽性率の差があるか。

RQ2 観点別並列の優位は、同コストの Multi-run と比べても残るか。

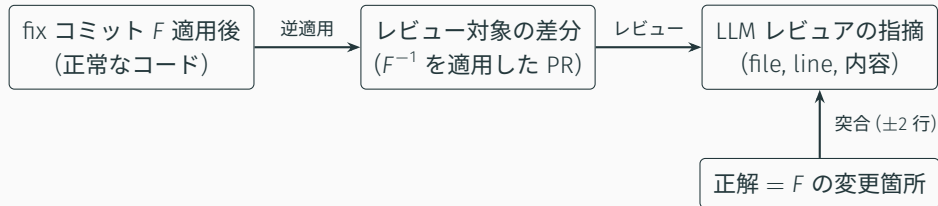
RQ3 欠陥の種別によって有利な構成は異なるか。

本発表の結論：予算帯ごとに最も高い F_2 を出した組み合わせ

項目あたり予算	最良の (モデル / 構成)	含意
\$0.002	Haiku 4.5 / Single	追加呼び出しの費用が支配的
\$0.01	GPT-5 mini / Adversarial	オーケストレーションが逆転
\$0.05 以上	GPT-5 mini / Multi-run	廉価モデルが Opus 5 単体を上回る

12 モデル中 9 モデルは、全 4 構成がパレートフロントから脱落した。

2. fix コミットの逆適用で、正解が一意的な課題を合成する



fix が追加した行を削除した差分を PR として提示し、その見落としを指摘できるかを評価する

- ・ 全リポジトリ・全バグに一律の手続きを適用でき、正解も一意に定まる。
- ・ 限界：合成 PR であり、実在した PR ではない。

2.4 言語 85 件，うち pgrust はカットオフ後だけの統制群

対象リポジトリと採用件数

リポジトリ	言語	件数
fastapi	Python	25
valibot	TypeScript	20
chi	Go	15
pgrust	Rust	25
計		85

- ・ リポジトリ効果と言語効果を分離するため，各リポジトリを単一言語で揃えた．
- ・ pgrust は全履歴が対象モデルのカットオフ後 → **leakage-free** な統制群．
- ・ 境界を 2026-01-01 とし，カットオフ前後を共変量として保持 (post 49 / pre 36)．
- ・ 分類は Claude 系，検証は異系統 LLM (GPT 系) 17 件で 12 件一致．

構築：候補 1,544 → 機械フィルタ 816 → LLM 分類 201 → 逆適用可 135
→ 層別抽出 85 件

3. Multi-run を「構造を持たない計算量増加」の対照群に置く

比較する 4 構成と、1 件あたりの LLM 呼び出し回数

構成	手続き	呼び出し
Single	単一レビューが全観点を担当	1 回
Parallel	4 観点 (correctness, concurrency, api-misuse, security) を独立実行し和集合	4 回
Adversarial	指摘生成の後、各指摘に反証プロンプトを 2 回適用し半数以上の支持で採用	$1 + 2k$ 回
Multi-run	同一プロンプトを 5 回実行し、2 票以上を採用	5 回

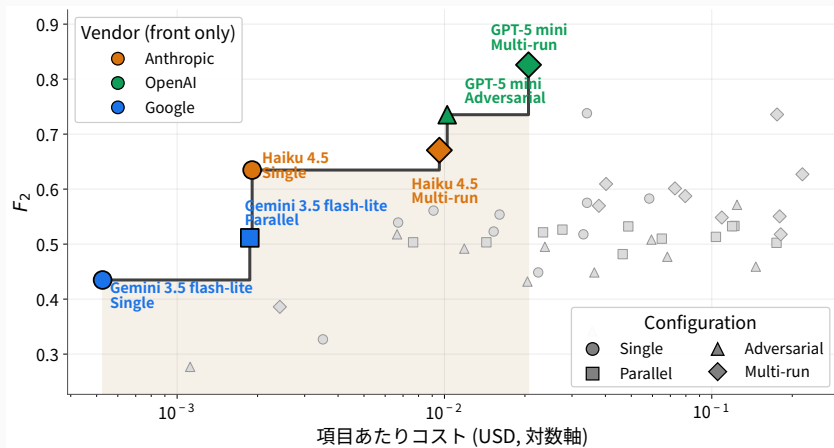
- ・ システムプロンプトの基底は全構成で共通、出力は {file, line, severity, category, summary} の JSON 配列.
- ・ Multi-run は構造を持たない計算量増加であり、RQ2 の対照群となる.

3. 見逃しの方が対処コストが高いので F_2 を主指標に置く

$$F_2 = \frac{5PR}{4P + R} \quad (\text{再現率 } R \text{ を適合率 } P \text{ の } 4 \text{ 倍 重視})$$

- ・ 誤指摘の却下はレビューの確認で済むが、見逃したバグの発見には人手レビューを要する。この非対称性に合わせ、見逃しを減らす側に重みを置いた。
- ・ 検出判定：指摘の (file, line 範囲) と正解ハンクの重なり (± 2 行) を機械判定する。
- ・ 規模：3 ベンダ 12 モデル $\times$ 4 構成 $\times$ 30 件 = 1440 実行，総 API 支出 \$79.79.

4. 12 モデル中 9 モデルは全構成が脱落した



項目あたりコスト (対数軸) と F_2 の散布. マーカ形状が構成, フロント上の点のみベンダ色で示す

4. Multi-run が全 β 帯で最高，Parallel の優位は残らない

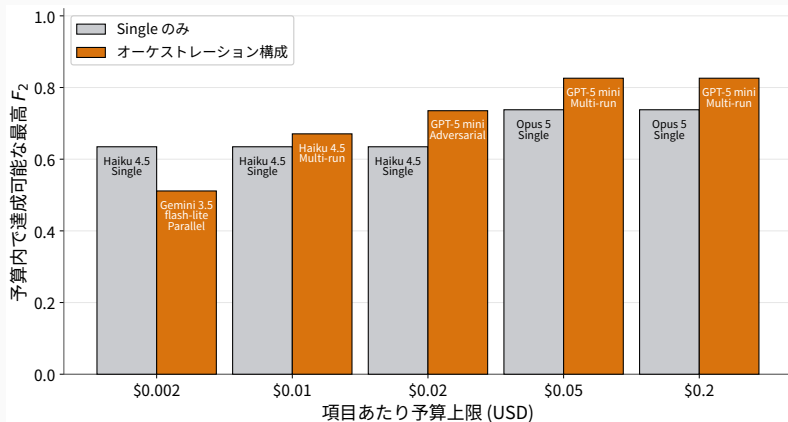
構成別の平均 F_β

Config	F_1	$F_{1.5}$	F_2	F_3
Single	0.619	0.563	0.536	0.514
Parallel	0.424	0.477	0.514	0.556
Adversarial	0.572	0.508	0.479	0.456
Multi-run	0.665	0.623	0.603	0.585

β を 1~3 に振っても Multi-run は全帯で最高

- 平均 F_2 の順位は Multi-run > Single > Parallel > Adversarial.
- Adversarial：反証で適合率は上がるが，本物のバグまで却下して再現率が下がる.
- Multi-run：2/5 のしきい値が適合率と再現率の双方を伸ばす.
- RQ2 への答え：観点別並列の優位は，同コストの計算量増加に対して残らない.

4. \$0.01 を境にオーケストレーションが Single を逆転する



予算上限ごとの最高 F_2 . \$0.05 以上では GPT-5 mini / Multi-run が Opus 5 単体を含む全 Single を上回り、予算を 4 倍にしても天井は上がらない

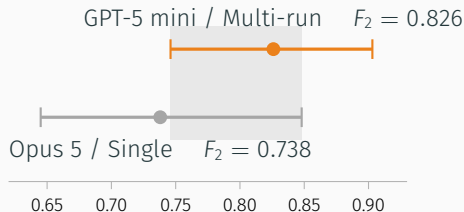
4. バグ種別ごとに最良の (モデル, 構成) は入れ替わる

バグ種別ごとの最高 F_2

バグ種別	最良 (モデル / 構成)	F_2	N
logic	GPT-5 mini / Multi-run	0.779	15
type-safety	Haiku 4.5 / Single	1.000	2 [†]
error-handling	Haiku 4.5 / Multi-run	1.000	1 [†]
concurrency	Haiku 4.5 / Adversarial	0.667	3
api-misuse	GPT-5 mini / Multi-run	1.000	4
security	GPT-5 mini / Adversarial	0.833	1 [†]
resource-leak	GPT-5 mini / Multi-run	0.897	2 [†]
other	GPT-5 mini / Adversarial	1.000	2 [†]

- ・ 最良は種別ごとに異なり，
GPT-5 mini (5 種別)，Haiku 4.5 (3 種別) に分かれる．
- ・ 最難は concurrency (最良でも $F_2 = 0.667$)．
- ・ 言語別では 4 リポとも GPT-5 mini / Multi-run が最良だが，
達成値は 0.722 (Python) ～ 0.972 (Go) と幅がある．
- ・ 種別あたり N は 1～15 件と小さく，
予備的示唆にとどまる．

4. 単一組の有意差は示せない．支えるのは横断的な一貫性



項目 bootstrap による F_2 の 95% 信頼区間．灰色の帯は 2 つの区間が重なる範囲を表す

- ・ $N = 30$ の本実験では，**単一の組どうしの有意差は主張しない**．
- ・ 支えるのは，モデル別集計で 12 モデル中 8 が Multi-run 最良という**一貫性**と， $\beta \in \{1, 1.5, 2, 3\}$ で順位が変わらない**指標の頑健性**である．

5. 結果の一般化を妨げる制約

- ・ **合成 PR**：逆適用 PR は実在した PR ではなく、レビュー体験の再現性に限界がある。
- ・ **位置一致のみの判定**：(file, line) の重なりで判定しており、意味的に等価な指摘を取りこぼしうる。
- ・ **標本サイズ**： $N = 30$ ，かつバグ種別で均等化しておらず個別順位の信頼性は低い。
- ・ **反証プロンプトの設計依存**：Adversarial は REFUTED 寄りに設計しており，再現率低下の一因の可能性がある。
- ・ **対象の限定**：商用 API 12 モデル，4 リポジトリ 4 言語に限る。
- ・ **予算統制は事後分析**：予算上限を強制したのではなく，実測コストを予算帯に集計して比較している。

5. 構造を足す前に，同コストの反復を対照群に置くべき

1. 構造の効果は，計算量と分離して検証しなければならない。
役割分担 (Parallel) や反証 (Adversarial) は，
同コストの単純な反復集約に対して優位を示さなかった。
2. 最適な構造は資源制約の関数であり，固定して選べない。
同じ予算で「高性能モデル 1 回」と「廉価モデル 5 回」のどちらが勝つかは，予算帯で
入れ替わった。
3. タスクを見て構成を切り替えるメタ制御が要る。
バグ種別によって最良の (モデル, 構成) が入れ替わり，単一の万能構造では届かない。

コードレビューは正解が機械判定でき，コストが API 課金で正確に測れる。
人手評価に頼らず予算-性能のトレードオフを反復実験できる，
AGI 研究の実験場として扱える領域である。

5. まとめ

- ・ バグ修正コミットの逆適用により、コストと検出性能を同一軸で比較できる欠陥検出ベンチマークを構築した (4 言語 85 件).
- ・ 1440 実行の実測から、
予算帯に依存して最適構成が入れ替わるパレートフロントを提示した.
- ・ 構造化された構成 (Parallel, Adversarial) の優位は、同コストの単純な反復集約に対して残らなかった.
- ・ 今後：意味一致判定器の導入、N の拡大と種別均等化、
複合構成および動的ルーティングの設計比較.

ベンチマーク・実験ハーネス・全実行記録：<https://github.com/hiroaki222/review-orchestration-bench>

付録：パレートフロント上の 6 点

コスト- F_2 パレートフロントを構成する 6 つの (モデル, 構成)

Model	Config	\$/item	F_2
Gemini 3.5 flash-lite	Single	\$0.0005	0.435
Gemini 3.5 flash-lite	Parallel	\$0.0019	0.511
Haiku 4.5	Single	\$0.0019	0.635
Haiku 4.5	Multi-run	\$0.0096	0.671
GPT-5 mini	Adversarial	\$0.0102	0.735
GPT-5 mini	Multi-run	\$0.0206	0.826

12 モデル $\times$ 4 構成の全 48 セル (適合率 $\cdot$ 再現率 $\cdot F_2 \cdot$ 項目あたりコスト) は論文および公開リポジトリの [data/experiments/v0/](#) を参照.

付録：言語別の最良構成

リポジトリ (言語) ごとの最高 F_2 とその組み合わせ

リポジトリ (言語)	最良 (モデル / 構成)	F_2
Python (fastapi)	GPT-5 mini / Multi-run	0.722
TypeScript (valibot)	GPT-5 mini / Multi-run	0.777
Go (chi)	GPT-5 mini / Multi-run	0.972
Rust (pgrust)	GPT-5 mini / Multi-run	0.909

- ・ 4 リポジトリすべてで GPT-5 mini / Multi-run が最良だった.
- ・ 達成 F_2 の幅は, 言語・ドメインが検出難度を左右することを示す.

付録：コストの内訳

- ・ 項目あたりコストは最安 \$0.0005 から最高 \$0.2182 まで 415 倍の広がりを持つが、 F_2 はコストに単調ではない。
- ・ 構成別の平均コスト：Single \$0.0196, Parallel \$0.0628, Adversarial \$0.0453, Multi-run \$0.0939.
- ・ 総 API 支出 \$79.79. 全 API 呼び出しの記録 (runs/*/ledger.jsonl) を公開しており、第三者によるコスト再計算が可能である。

- [1] Xunzhu Tang et al. *CodeAgent: Autonomous Communicative Agents for Code Review*. 2024.
- [2] Taicheng Guo et al. *Large Language Model based Multi-Agents: A Survey of Progress and Challenges*. 2024.
- [3] Tian Liang et al. “Encouraging Divergent Thinking in Large Language Models through **Multi-Agent Debate**”. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Ed. by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Association for Computational Linguistics, Nov. 2024.
- [4] Noah Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023.
- [5] Qineng Wang et al. *Rethinking the Bounds of LLM Reasoning: Are Multi-Agent Discussions the Key?* 2024.